Bd resumo

Dependcias funcionales:

Dependencia Parcial: Un atributo depende de solo *una parte* de los atributos que componen la clave de la relación.

Dependencia Transitiva: Un atributo que no es clave depende de *otro atributo que tampoco es clave*.

Dependencia Total: El atributo depende de *toda* la clave de la relación

NORMALIZACION

1FN (Primera Forma Normal): Establece que todos los atributos deben ser **atómicos** (simples e indivisibles). **No se permiten atributos multivalorados** ni relaciones anidadas (relaciones dentro de relaciones).

2FN (Segunda Forma Normal): La tabla debe estar en 1FN y **no deben existir dependencias parciales**. Todos los atributos que no pertenecen a ninguna clave deben depender *totalmente* de la clave primaria entera.

3FN (Tercera Forma Normal): La tabla debe estar en 2FN y **no deben existir dependencias transitivas**. Ningún atributo no clave debe ser determinado por otro atributo no clave.

BCNF (Forma Normal de Boyce-Codd): Es una versión más estricta de la 3FN. La regla es: **"Todos los atributos son funcionalmente dependientes de la clave de la relación, de toda la clave y de nada más"**. Sin embargo, el resumen advierte que al pasar de 3FN a BCNF **se puede perder alguna dependencia funcional útil**, por lo que el "punto de equilibrio" en el diseño a menudo se queda entre la 3FN y la BCNF.

4FN y 5FN: Generalmente, si alcanzas la BCNF, ya estás en 4FN y 5FN. La 4FN exige que no haya **dependencias multivalor**, mientras que la 5FN exige que no haya **dependencias de junción** (un concepto muy raro en la práctica donde una tabla solo puede reconstruirse uniendo tres o más proyecciones específicas

2) INDEX , b trees page Split

single level index

primary index , no se repite , clustered puede tener duplicados , los atributos están agrupados

Solo podemos tener primarias o clustered

Multivariable index : varios niveles de indexación por log

Arboles de pesquisa , cada no contem maximo p – 1 valores e p ponteiros key value tiene associado un puntero para el registro

Se dice balanceada si la distancia de cualquier hoja a la raíz es la misma , los nos están en el mismo nivel

Un índice puede acelerar el proceso de búsqueda por un valor en atributo

Pero puede porvocar overhead , en el volumen de datos , y del tiempo de inserción

La esoja debe ser entre la necesidade buscar algo en el index , guardados en paginas

Si el índice no es la llave de la relación se tiene pocos valores repetidos

La escoga de índices por costos

En sql funciona como clustered o non clustered todas con balanced trees

Cuando queremos hacer insert en algo ya lleno se hace un page Split , crear nueva pagina , copia algunas cosas para allá , el page Split es penalizador en desempeño temoral

Una llave primaria crea una unique cluestered index ,

Heap vs clustered , heap agrega en el final de la tabla , una clustered , instroduce el nuevo registro en la B tree segundo la cluster index key

Chave primaria , unique , evitar llaves suceptibles a crear page Split

Llaves pequeñas mejor

Se pueden mejorar la b tres para impedir el page Split

Para desfragmentar los índices debemos , verificar el estado del fragmento del índice , reconstruir caso este afectado ,

3) SQL programming

Batch grupo de instrucciones lógicas , si una erra todo erra m un error de runtime no mata todo

Se udas después son eliminadas todas las locales , tabelas e cursores creados ,

Script son ficheros de texto delimitadas por GO , las barch son usadas en secuencia

Print imprime en la consola , print ‘ola’ , declare @ temp int = 5; . print ‘ temp value: ‘ + str()

@temp

;

Begin end bloque de instrucciones

If else boolean

While boolean statement , block y eso

Case when

Tabelas de uso temporario , tienes caracteristicaa como las normales pro falta , las locales tienen # antes del nombre , solo se ven es ;a sesión que la creo y en el nivel en que los creo las inner level , son eliminadas cuando la función termina

Las golbales ## , tempdb tiene scope mayor ,

Tablas como variables , scope mas limitado , no son visibles por inner levels de la call stack

Desaparecen cuando la batch procedimiento o función llega a su fin no permiten llaves extranjeras y checks

Cursor : peromite recorer las secuencias en tuplos por los select set based mas rápido que cursor declare,open,fetch,close,deallocate

SP : batch con un nombre , compila de una vez es una batch , el servidor no tiene que recopilar informacoin cada vez que se compila , los procedures co guardados en cache cuando son ejecutadas

Tipos : system sp , nombre comienza por uno , creados en master base , usados e qualqer base de datos

Local sp : son definidos en una base de datos local , nombre libre una normalización por parte del que la usa

La sp debe tener una variable antes para poder salir , terina y retorna un entero , nomralmene succes p failure , valor de salida puede ser atribuido a una variable

T sql permite mandar errores

UDF user defined functions : mismos beneficios de los stored procedures , pueden tene runa lógica compleja , 3 tipos

Escalares : aceptan varios parámetros , retornan un único valor , instrucción return , pueden ser usados dentro de cualquier t sql o check constrait , tienen limitaciones determinísticas ósea el mismo valor de entrada puede tener el mismo valor de retorno , no son permitidas funciones no determinísticas dentro de una udf

No permiten updates , no permite retornar , binario , text , image , etc , table variables y cursor , recursividad limitada a 32 niveles

Inline table valued: soportan parámetros de entrada , casi igual

Multi statement table valued :comnina function escalar , codigo Complejo con la capacidad de retornar un conjunto como la inline table value , crea una table variable introduz tuplos y se retorna

SP versus UDF

• return - zero, single or multiple values	• return - single value (scalar or table)
• input/output param	• input param
• cannot use SELECT/ WHERE/ HAVING statement	• can use SELECT/ WHERE/ HAVING statement
• call SP - OK	• call SP - NOK
• exception handling - OK	• exception handling - NOK
• transactions - OK	• transaction - NOK

Trigger : tipo especial de sp , que se ejecuta en determinadas eventos asociado con los datos , sql suporta dml o ddl , triger dml: son creados en tabelas insert update delete

Cuando ocurre lo necesario se dispara , ocurren , son disparados para cada modificación de datos , pueden causar retrasos o integridad , instead of after ,

- **AFTER Trigger:** Se ejecuta *después* de que los datos pasaron las verificaciones de constraints. Sin embargo, se ejecuta *antes del COMMIT* final, lo que te permite hacer un ROLLBACK (cancelar y deshacer la operación) si los datos violan reglas de negocio complejas.
- **INSTEAD OF Trigger:** Solo puede haber uno por tabla/vista. **No se ejecuta la acción original** (el Insert, Update o Delete se cancela), y en su lugar se ejecuta únicamente la lógica del trigger. Es súper útil para actualizar "vistas" que normalmente no permitirían actualizaciones o para desviar registros eliminados a una tabla de histórico

Tablas Lógicas de los Triggers (Inserted **y** Deleted**)** Dentro de un trigger, el sistema te da acceso a dos tablas temporales mágicas:

- Inserted: Contiene los registros nuevos (en un INSERT o la nueva versión en un UPDATE).
- Deleted: Contiene los registros viejos (en un DELETE o la versión anterior en un UPDATE)

Triger no permite crear alterar o drop database , reconfigura , restore la database or log , disk resize y disk init

4) sgbd intermdeio entre app y la bd es multi utilizador

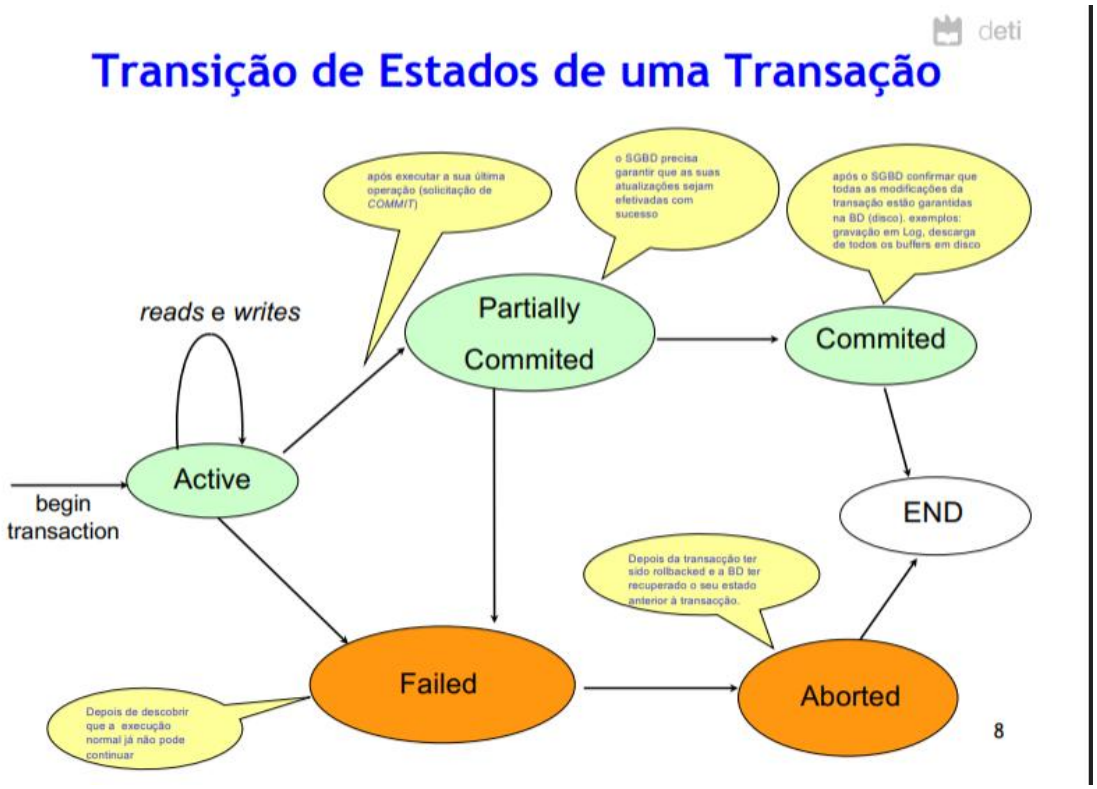
La transacción : unidad lógica de trabajo que contiene una o mas operaciones tiene operaciones de read (transfiere el elemento x de la base de datos para la área de memoria volátil associada a la transacción que hizp la lectura) y de write(trasfiere x para la memoria que afecta la transacción de la base de datos)

Ósea read(a) a a – 50 write a , asi lee y guarda en la memoria

Un comando individual como un delete puede ser considerado una transacion ósea DELETE FROM Pacientes

Begin transaction , suceso commit , falha rollback

el estado son controlados por el sgbd



Propiedades :

Atomicidad : las operaciones de la transacción ocurren de forma atómica , o todas o ninguna , o todo o nada , el sgbd es responsable por recuperar faltas

Consistencia : debe enviar las cosas de forma integra

Isolamento : es deseable que puedan ser echan de forma concurrente de forma isolada

Persistencia : los efectos de una transacción terminada con commmit son permaenetes y visibles para las otras

el sgbd debe garantir que las modificaciones sean echan son sucesso

Concorrencia : garantina de isolamento : escalonmento serie : de forma secuencial y una solución bastante ineficiente

Escalonamiento concorrente serie : ejecución concorrente pero preserva el isolamiento obliga a resultados equivalentes

Escalonador : entidad responsable de definit escalonamientos de transacciones , un detemrinado momento e , define una ejecución intercalada problemas puede ser actualización perdida , lectura sucia

Métodos de control de concorrencia :

Mecanismo etiquetagem : preventivos y el objetivo es permitir transacciones hasta donde sea posible , y evitar operaciones que provoquen interferencias en transacciones

Se le da una etiqueta , pero si intenta entrar a una transacción que inicio mas tarde la transacción se cancela y reiniciada

Mecanismos Locking : metdodo que lock es una variable asociada a determinado elemento de la base locked binarios 1 locked 0 unlocked , lock lecturas / escritas . solo accede a escrita

Recuperar fallas : falla menos grave falla una transacción , grave perda de la base de datos , recuperar : escalonamiento, backups , transaction logging

Un escalonamiento recuperable , si ninguna ti for concluida hasta que los elementos hayan sido concluidos , pueden ser en cascata

Estricto : si e puede leer o analizar depues de que las transacciones hayan acabado

Backup : copia de seguridad , se puede hacer una T log para intentar guardas cosas

El data Flow , donde el log se escribe backup + transaction logs

Rollforward hasta las parte donde se puede , normalmmento el t4 no se llega completo dado que fue donde ocurrió la falla

Rollback seguo el ultimo backup solución baseada e chekpoint

Puedes tener un sabe point , la recuperación tiene fallas y costos

En sql seria begin t , commit , rollback , save point

NOSQL: cosas separadas normal , duplicados desnormalizado

Los data trends

No sql , no relaiconal , simple api , base & cap theorem , , un esquema libre , sitribuido , y open source .

Acid es un banco seguro , BASE es una conversación libre en una red social

Teorema de brewer : un sistema de distribución que puede soportar características , consistente , permitida y con partición tolerable

CP error , AP , bien

Las key values data : en sesiones de data , user profiles , no se usan en entidades de relación o en queries que requieren una parte

Bases de datos en columnas buenas para queries que involvan pocas columnas , vast amount o f data

No muy buenas en la data incremental

ventajas si : algunas pueden ser muy rapudas , mejor compresión de datoa

des : la agregación es buena pero algunas apps necesitan mostrar la data para cada record , escribir datos puede tardar mucho

columna databases : con muchas cosas no es buena idea

se usan tipos de grafos : de relación única , o multi relacionales

se usan es cosas bien normal , pero cuando hay muchas batch no es buena idea

Time series : modelo de dato para uso en curvas perfiles y eso

1. ¿Por qué surge NoSQL? El problema de escalar Las bases de datos relacionales son geniales, pero tienen problemas fundamentales para manejar los volúmenes gigantescos de datos actuales.

- **Escalabilidad Horizontal vs. Vertical:** Para que una base de datos relacional crezca, normalmente necesitas una máquina más grande y cara (escalado vertical). NoSQL está diseñado para escalar horizontalmente, es decir, repartir los datos en clústeres de miles de máquinas pequeñas y baratas.
- **Impedance Mismatch (Pregunta de examen):** Es el problema de que nuestro código usa "objetos" complejos, pero las bases relacionales nos obligan a desarmar y normalizar esos objetos en múltiples tablitas separadas matemáticamente. NoSQL soluciona esto permitiendo guardar la estructura completa de una sola vez.

2. Adiós a ACID, hola a BASE y CAP Theorem ⚠️ ¡CRÍTICO! En el tema anterior viste que las relacionales usan transacciones **ACID** (estrictas, como la bóveda de un banco). En NoSQL, las reglas se relajan para ganar velocidad y disponibilidad usando el modelo **BASE** (como una red social):

- **B**asic **A**vailability: El sistema casi siempre responde.
- **S**oft-state: El estado no siempre tiene que ser perfectamente consistente en todas las réplicas en todo momento.
- **E**ventual consistency: El sistema alcanzará la consistencia "eventualmente" (más adelante).

El Teorema CAP (Teorema de Brewer): Establece que un sistema distribuido solo puede tener 2 de estas 3 características al mismo tiempo:

1. **C**onsistency (Consistencia: todos leen el mismo dato nuevo).
2. **A**vailability (Disponibilidad: el servidor siempre responde si está encendido).
3. **P**artition Tolerance (Tolerancia a particiones: el sistema sobrevive si se cae la red entre nodos). *Ojo aquí:* Como escalar horizontalmente obliga a particionar los datos (P), los diseñadores de NoSQL siempre tienen que **elegir sacrificar la Consistencia (C) o la Disponibilidad (A)**.

3. Los 4 Tipos Principales de Bases de Datos NoSQL NoSQL no es una sola tecnología, se divide en 4 modelos "Core" que debes saber diferenciar:

- **Clave-Valor (Key-Value):** Es un simple diccionario o *hash table*. Extremadamente rápido y escalable. El valor es una "caja negra" para el sistema. *Casos de uso:* Sesiones de usuario, carritos de compra, preferencias. *Ejemplo:* Redis.
- **Documentales:** Guardan datos estructurados en formato jerárquico como JSON o XML. La gran diferencia con Clave-Valor es que aquí **la base de datos sí entiende los datos internos**, permitiendo hacer consultas y crear índices sobre los atributos internos. Suelen estar desnormalizadas (guardan documentos anidados). *Casos de uso:* Blogs, catálogos e-commerce, CMS. *Ejemplo:* MongoDB.
- **Orientadas a Columnas (Column Stores):** En lugar de guardar filas enteras juntas, agrupan y guardan los datos por columnas. *Ventaja:* Excelentes para comprimir datos y hacer consultas de agregación masivas (como promedios o sumas en Big Data). *Desventaja:* Son muy lentas para hacer inserts fila por fila (OLTP). *Ejemplo:* Apache Cassandra.
- **Grafos (Graph Databases):** Modelan los datos como Nodos (entidades) y Aristas (relaciones directas). Ambos pueden tener propiedades. *Casos de uso:* Redes sociales (quién conoce a quién), motores de recomendación y rutas. *Ejemplo:* Neo4j. *(Nota adicional: También existen bases de datos de **Series de Tiempo**, como InfluxDB, ideales para sensores, señales de ECG o datos de bolsa)*.

4. Conclusión Estratégica: "Persistencia Políglota" El profesor aclara algo importante: **NoSQL no es el fin de las bases relacionales** (de hecho, el 90% de los proyectos aún las usan por su estabilidad). El concepto moderno de diseño es la **Persistencia Políglota**: usar distintos tipos de bases de datos para resolver distintos problemas dentro del mismo proyecto